



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Impiego dei Large Language Models nella Code Review basata su Information Needs

RELATORE

Prof. Fabio Palomba

Dott. Gilberto Recupito

Università degli Studi di Salerno

CANDIDATO

Giulio Palladino

Matricola: 0512106797

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

I learned a lot, by the end of everything. The past is past, now, but that's... you know, that's okay! It's never really gone completely. The future is always built on the past, even if we won't get to see it. Still, it's um, time for something new, now.

-Riebeck, Outer Wilds

Abstract

La Code Review tradizionale è un'attività cruciale ma costosa, suscettibile a errori umani e limitata nella sua capacità di soddisfare tutti i bisogni informativi (Information Needs) necessari per una revisione completa. Gli approcci esistenti di automazione tendono a focalizzarsi su aspetti tecnici circoscritti, tralasciando spesso le dimensioni contestuali più sottili. L'utilizzo dei Large Language Models (LLM) è sempre più integrato nelle diverse fasi dello sviluppo software. Questa tesi si concentra sul loro impiego nella fase di Code Review, con particolare attenzione alla capacità dei modelli di soddisfare delle esigenze informative specifiche che emergono durante l'analisi del codice da parte degli sviluppatori, dette Information Needs. Lo studio confronta le Code Review condotte da sviluppatori umani, estratte da repository GitHub, con quelle generate da un tool basato su un LLM istruito per tenere conto degli Information Needs. L'obiettivo è analizzare quali Information Needs vengano soddisfatti nei due approcci e in che modo vengano affrontati. I risultati mostrano che le review prodotte dal tool LLM CoRALINE coprono sistematicamente tutti gli Information Needs identificati, ma tendono a fornire suggerimenti più generici e meno contestualizzati. Le revisioni umane, invece, pur non coprendo l'intero spettro dei bisogni informativi, risultano più mirate, specifiche e orientate alla risoluzione concreta di uno specifico problema. Lo studio evidenzia inoltre una limitazione significativa legata alla capacità del modello di gestire il contesto: a causa del limite nella finestra di input, non è stato possibile fornire informazioni complete, compromettendo in parte la profondità dell'analisi automatica. I risultati suggeriscono che gli LLM possono essere un valido supporto alla Code Review, ma richiedono ottimizzazioni per un uso realmente efficace in scenari reali.

Indice

1	Introduzione	1
1.1	Contesto	1
1.2	Obiettivi	2
1.3	Struttura della tesi	2
2	Background e Stato dell'Arte	4
2.1	Background	4
2.1.1	LLM	4
2.1.2	Code Review	5
2.1.3	LLM e Code Review	5
2.1.4	Information Needs	6
2.2	Stato dell'Arte	9
2.2.1	Agenti basati su LLM	9
2.2.2	Architettura degli agenti	9
2.2.3	Code Review con agenti LLM	14
3	Metodo di Ricerca	16
3.1	Domande di Ricerca	16
3.2	Pipeline di Ricerca	18
3.2.1	Raccolta e preparazione dei dati	18

3.2.2	Categorizzazione in base agli Information Needs	18
3.2.3	Analisi mediante tool LLM	19
3.2.4	Confronto dei risultati ottenuti	19
3.3	CoRALINE	20
3.3.1	Scelte progettuali	20
4	Analisi dei Risultati	25
4.1	Introduzione	25
4.2	Metodologia di analisi	25
4.3	Risultati sperimentali	26
4.3.1	RQ1. Quali sono gli Information Needs meno soddisfatti nelle Code Review umane?	26
4.3.2	RQ2. Per quali specifici Information Needs gli LLM dimostrano capacità più limitate?	27
4.3.3	RQ3. Quali sono i principali punti di forza e debolezze degli agenti basati su LLM nel soddisfare i singoli Information Needs e quali differenze emergono con le review umane?	29
4.4	Discussione critica	31
5	Conclusioni e Sviluppi futuri	33
	Bibliografia	35

CAPITOLO 1

Introduzione

1.1 Contesto

Negli ultimi anni i **Large Language Models** (LLM) hanno rivoluzionato numerosi ambiti dell'informatica, dimostrando potenzialità significative nelle diverse fasi dell'ingegneria del software grazie alla loro capacità di comprendere e generare codice in modo simile a un programmatore umano [1]. In particolare, l'impiego degli LLM ai fini di automatizzare il processo di **Code Review**, fase cruciale per garantire la qualità, sicurezza e manutenibilità del codice. La Code Review tradizionale è un'attività complessa e costosa. Richiede molto tempo e sforzo cognitivo da parte degli sviluppatori, è suscettibile a errori umani e incongruenze [2], e spesso trascura aspetti importanti per mancanza di risorse o attenzione. Per garantire la completezza dell'attività di Code Review, sono stati identificati specifici bisogni informativi (**Information Needs**) che i revisori devono soddisfare durante il processo. Proprio qui gli LLM possono offrire un contributo decisivo, aiutando a colmare queste lacune analizzando il codice in modo rapido e sistematico. Tuttavia, rimangono aperte importanti questioni: *in che misura questi modelli possono realmente comprendere e supportare tali esigenze? Forniscono suggerimenti generici o fuorvianti [3]? E, soprattutto, possono essere integrati nel processo senza comprometterne l'affidabilità?*

1.2 Obiettivi

Questa tesi si concentra sull'analisi sistematica dell'impiego degli LLM nella Code Review, ponendo particolare attenzione alla loro capacità di soddisfare gli Information Needs identificati in letteratura. Questo lavoro adotta una prospettiva strutturata, confrontando le revisioni umane con quelle automatizzate per ciascuna categoria di bisogno informativo. L'utilizzo degli LLM per automatizzare parzialmente il processo di Code Review potrebbe introdurre importanti benefici, come la riduzione del carico cognitivo dei revisori e dei tempi di analisi, oltre a una maggiore copertura di aspetti tecnici spesso trascurati. Gli obiettivi dello studio sono di comprendere quali Information Needs risultano meno coperti dall'approccio tradizionale. Inoltre, si intende analizzare dove gli LLM mostrano maggiori limitazioni, quali sono gli effettivi punti di forza degli agenti basati su LLM e le differenze tra le due metodologie. I risultati consentiranno di ridurre significativamente il carico dei revisori umani mantenendo elevati standard qualitativi, definendo le aree in cui l'impiego degli LLM può apportare massimo beneficio. La ricerca, oltre a contribuire al dibattito scientifico sull'affidabilità degli LLM in ambiti complessi, fornisce risultati che potranno guidare la progettazione di strumenti concreti a supporto dei revisori.

1.3 Struttura della tesi

Il seguente lavoro di tesi è organizzato in cinque capitoli. Il Capitolo 2 fornisce le basi teoriche, illustrando i concetti fondamentali sui LLM, il processo tradizionale della Code Review, la tassonomia degli Information Needs e una rassegna dello stato dell'arte sull'uso degli agenti basati su LLM in ambito ingegneristico. Il Capitolo 3 descrive nel dettaglio la metodologia adottata, dalla raccolta e categorizzazione delle revisioni umane, all'analisi condotta attraverso il tool basato su agenti LLM, fino alle modalità di confronto tra approccio umano e automatico. Il Capitolo 4 presenta i risultati sperimentali, analizzando sistematicamente le differenze emerse nel soddisfacimento degli Information Needs e discutendo punti di forza e limiti degli LLM. Infine, il Capitolo 5 sintetizza le conclusioni dello studio, propone raccomandazioni

pratiche per l'uso degli LLM nei processi di revisione e delinea possibili sviluppi futuri della ricerca.

CAPITOLO 2

Background e Stato dell'Arte

2.1 Background

In questo capitolo sono descritte le nozioni preliminari utili a comprendere il contesto della ricerca. La trattazione si articola in quattro sezioni principali che forniscono: (1) una panoramica sui Large Language Models, (2) i principi base della Code Review, (3) l'applicazione degli LLM in questo ambito, e (4) la tassonomia degli Information Needs. Questi concetti costituiscono la base teorica indispensabile per analizzare criticamente i risultati presentati nei capitoli successivi.

2.1.1 LLM

I Large Language Model sono programmi di Intelligenza Artificiale capaci di interpretare e produrre testi in linguaggio naturale. Sono addestrati utilizzando grandi insiemi di dati provenienti da diverse fonti e possono eseguire una varietà di compiti, come rispondere a domande, tradurre testi, generare immagini e scrivere codice. L'esempio più noto, al giorno d'oggi, è sicuramente ChatGPT. Grazie alle loro ampie capacità, il loro uso è sempre più diffuso. L'applicazione più nota è quella dell'IA generativa, in cui, alla ricezione di una domanda, viene prodotto del testo in

risposta. Anche i programmatori utilizzano tali strumenti per svolgere attività come la generazione di codice, il debugging o la code review.

2.1.2 Code Review

La code review è un passaggio fondamentale nello sviluppo di un software. Durante questa attività, dei revisori esaminano il codice prodotto da altri sviluppatori a scopo di garantirne la correttezza, la chiarezza e il rispetto delle linee guida stabilite dal progetto. Gli attori principali coinvolti nel processo sono l'autore delle modifiche e i revisori. Come descritto da Yang et al. [4], le fasi principali del processo di revisione del codice sono: creazione della modifica, assegnazione dei revisori, commenti, modifica e risposta, approvazione e merge nel ramo principale del progetto. Nonostante i vantaggi, il processo presenta limitazioni che possono influire sulla sua efficacia: richiede un elevato impiego di risorse, tra cui tempo e personale; inoltre, è un'attività soggettiva, coinvolge diversi giudizi e opinioni che possono portare a disaccordi tra i revisori.

2.1.3 LLM e Code Review

Lo studio condotto da Alami et al. [3] evidenzia come gli LLM possano alterare le dinamiche sociali e la responsabilità collettiva nelle revisioni del codice. I risultati mostrano che gli LLM interrompono il processo di responsabilità condivisa che caratterizza le revisioni tra esseri umani. Durante le revisioni tradizionali, gli sviluppatori passano da una responsabilità individuale a una collettiva, regolando fattori intrinseci come l'integrità professionale per favorire il consenso del gruppo. Al contrario, gli LLM, privi della capacità di reciprocità e interazione sociale, compromettono questo equilibrio. La mancanza di fiducia negli LLM è un altro fattore critico. Nonostante la qualità dei commenti generati, gli sviluppatori rimangono scettici sulla capacità degli LLM di comprendere il contesto del codice, limitandone l'adozione come sostituti completi delle revisioni umane. Tuttavia, gli LLM possono essere utili come primo filtro per identificare errori evidenti, lasciando ai revisori il compito di gestire aspetti più complessi e sociali. In sintesi, l'integrazione degli LLM nelle revisioni del codice richiede un approccio bilanciato che preservi le dinamiche

sociali fondamentali, utilizzando gli LLM come supporto tecnico senza sostituire l'interazione umana. Questo equilibrio è essenziale per mantenere elevati standard di qualità e responsabilità nello sviluppo del software.

2.1.4 Information Needs

Al fine di svolgere efficacemente l'attività di code review, ridurre il tempo necessario e migliorarne la completezza, Pascarella et al. [5] hanno condotto uno studio volto a definire in modo empirico i bisogni informativi (Information Needs) dei revisori durante la revisione del codice. L'obiettivo della ricerca era quello di individuare possibili direzioni per future indagini nell'ambito della code review e contribuire allo sviluppo di strumenti di ingegneria del software in grado di migliorare la collaborazione e il coordinamento tra autori e revisori del codice. Gli Information Needs individuati sono stati analizzati e successivamente classificati in sette categorie distinte, in modo da offrire un quadro strutturato delle esigenze più ricorrenti riscontrate durante il processo di revisione. Tali categorie costituiscono una tassonomia utile per comprendere meglio il ruolo e le necessità dei revisori e per orientare l'evoluzione di strumenti e pratiche nel contesto collaborativo della code review. La Figura 2.1 rappresenta la suddivisione appena descritta. Descriviamo brevemente ogni categoria.

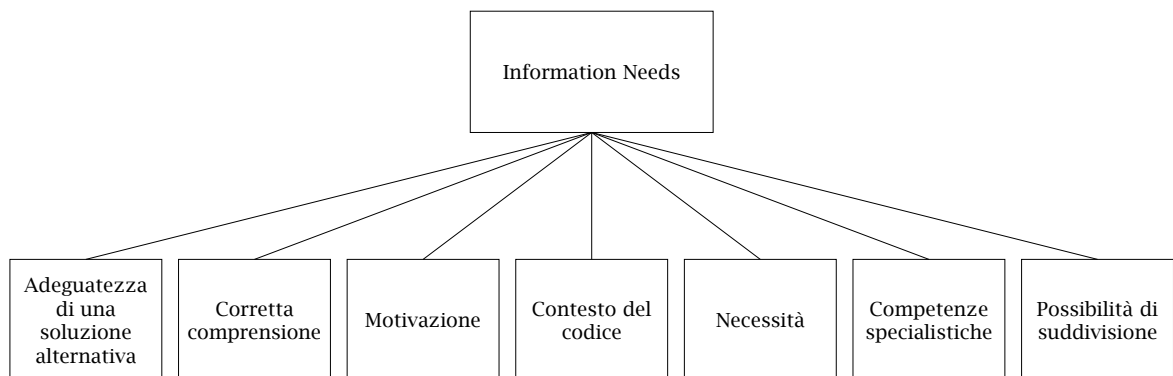


Figura 2.1: Classificazione degli Information Needs

Adeguatezza di una soluzione alternativa

Questa categoria comprende quei casi in cui il revisore avvia una discussione su possibili alternative all'implementazione proposta dall'autore. L'obiettivo non è solo valutare altre opzioni, ma anche stimolare un confronto che porti a miglioramenti del codice.

Corretta comprensione

Rientrano in questa categoria le domande con cui i revisori cercano di assicurarsi di aver interpretato correttamente le modifiche presentate. Tali domande sono spesso poste chiarire eventuali dubbi, soprattutto quando mancano commenti o documentazione.

Motivazione

Questa categoria fa riferimento alle richieste di informazioni che servono a comprendere il motivo per cui una certa modifica è stata proposta o implementata in un determinato modo. Il revisore chiede dettagli utili a capire se la modifica è coerente con gli obiettivi del progetto.

Contesto del codice

In questa categoria rientrano le domande inerenti a ottenere informazioni sul contesto in cui viene eseguito il codice modificato. Anche se il revisore ha accesso al codice, potrebbe aver bisogno di chiarimenti aggiuntivi per comprendere meglio le scelte fatte dagli autori.

Necessità

Qui si raccolgono le domande che mirano a capire se una parte della modifica sia realmente indispensabile o se possa essere semplificata o eliminata, ad esempio in presenza di codice duplicato o superfluo.

Competenze specialistiche

Questa categoria riguarda i casi in cui il revisore identifica un problema, ma non possiede le conoscenze tecniche per suggerire una soluzione. In questi casi,

coinvolge altri revisori con competenze specifiche o richiede all'autore di proporre possibili alternative.

Possibilità di suddivisione

Si tratta delle domande che cercano di capire se le modifiche proposte possano essere divise in più parti distinte e valutate separatamente, in quanto modifiche atomiche e auto-contenute aiutano a gestire meglio il carico cognitivo dei revisori e a rendere più chiaro ogni singolo intervento.

2.2 Stato dell'Arte

Gli argomenti trattati in seguito si basano sugli studi di Liu et al. [6], che analizzano in modo approfondito l'applicazione degli agenti basati su LLM nell'Ingegneria del Software.

2.2.1 Agenti basati su LLM

Gli LLM sono sempre più applicati nell'Ingegneria del Software (SE), grazie alle loro prestazioni in compiti come sviluppo del software, debugging, testing, generazione del codice e altre attività correlate. In combinazione con gli LLM vengono utilizzati degli agenti AI, entità artificiali capaci di percepire e agire in modo autonomo con l'ambiente circostante al fine di raggiungere uno scopo specifico. Nascono così gli agenti AI basati su LLM, cioè agenti che utilizzano gli LLM come controller centrale. A differenza dei LLM standalone, questi agenti sono in grado di percepire e usare risorse e strumenti esterni, ampliando così la versatilità e la capacità di affrontare obiettivi più complessi. Gli LLM standalone sono specializzati nell'eseguire compiti specifici di SE, ma si dimostrano inadeguati per compiti complessi. Gli agenti basati su LLM grazie ai loro componenti, alla coordinazione tra più agenti e all'interazione umana, offrono maggiore autonomia e flessibilità. È possibile distinguere tra sistemi single-agent e sistemi multi-agent. Nel primo caso, abbiamo un singolo agente specializzato nella risoluzione di un singolo compito; nel secondo, abbiamo diversi agenti, ciascuno con un ruolo, competenze e compiti diversi che possono anche collaborare tra loro per condividere informazioni. Secondo le statistiche di Liu et al.[6] il 52,8% degli agenti esistenti per l'ingegneria del software sono sistemi multi-agente. Alcuni agenti adottano un paradigma di cooperazione uomo-agente al fine di evitare colli di bottiglia e migliorare le prestazioni grazie all'expertise umana. In particolare, tale interazione avviene durante le fasi di pianificazione, requisiti, sviluppo e valutazione.

2.2.2 Architettura degli agenti

Come descritto da Liu et al. [6] gli agenti basati su LLM sono generalmente composti da quattro componenti chiave: pianificazione, memoria, percezione e azione.

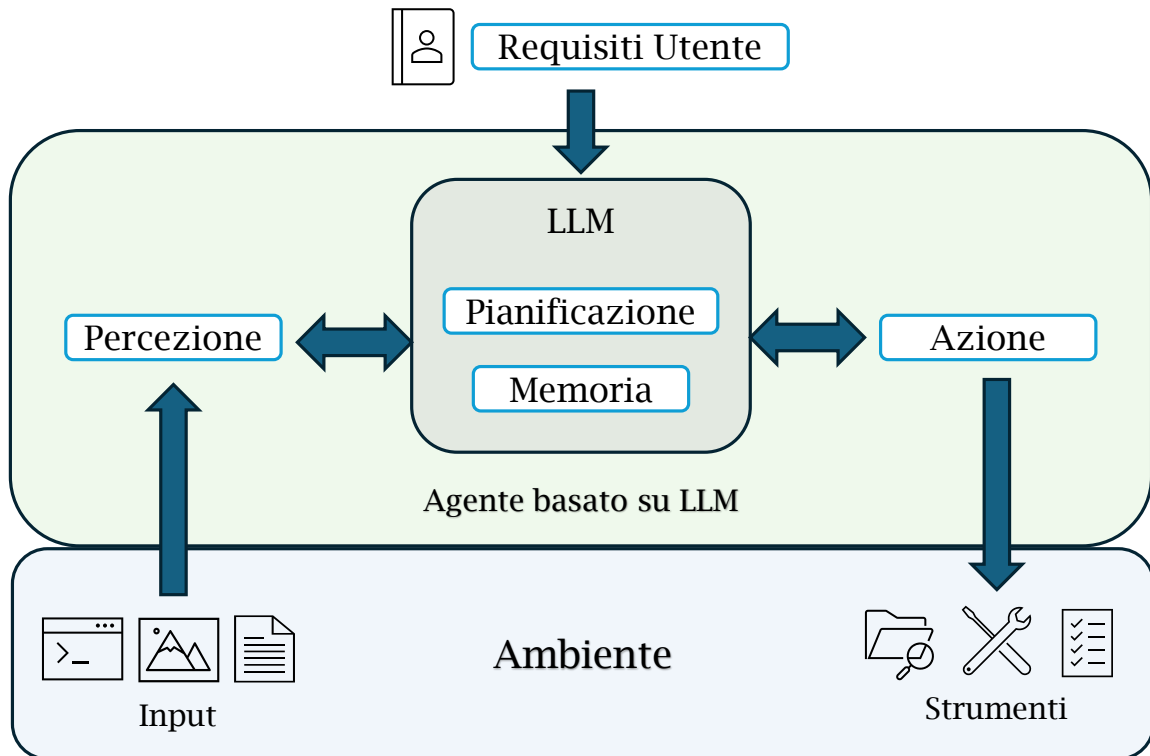


Figura 2.2: Esempio di un' architettura a un singolo agente basato su LLM

Nella Figura 2.2 viene illustrata l'architettura di base.

A. Pianificazione

Nelle attività di SE è necessario il coordinamento tra i vari agenti. La pianificazione consente di definire le sequenze di compiti, scomporre quelli più complessi in sotto-compiti e definisce l'ordine di esecuzione di tali compiti al fine di garantire una progressione fluida ed efficiente.

A.1. Singolo Pianificatore vs Pianificatori Multipli

La pianificazione è di solito gestita da un singolo agente, esistono tuttavia approcci in cui, con l'obiettivo di aumentarne l'accuratezza, essa è gestita da diversi agenti.

A.2. Pianificazione a Singolo Turno vs. Pianificazione a Turni Multipli

La soluzione maggiormente adottata consiste nel creare un piano globale ed eseguirlo in fasi successive. Alcuni agenti, invece, non procedono alla fase successiva fino a quando non ricevono un feedback da quella precedente, in modo da ottenere una maggiore flessibilità.

A.3. Pianificazione a Percorso Singolo vs. Pianificazione a Percorsi Multipli

La strategia principale è la pianificazione a percorso singolo, cioè gli agenti pianificano ed eseguono i compiti in modo lineare. Nella pianificazione a percorsi multipli vengono generati diversi piani che sono successivamente selezionati, cambiati o aggregati per garantire una pianificazione ottimale.

A.4. Rappresentazione del Piano

Esistono diverse modalità con cui è possibile rappresentare un piano, tra cui grafi, linguaggio naturale e rappresentazioni semi-strutturate.

B. Memoria

La componente di memoria registra informazioni come pensieri, osservazioni e azioni effettuate dall'agente durante la sua esecuzione e consente di rivisitare e sfruttare tali informazioni durante fasi diverse, in modo da garantire una maggiore efficacia ed efficienza, mantenendo un ragionamento coerente durante lo svolgimento dei task. La sua implementazione presenta quattro caratteristiche chiave: proprietà, durata, operazioni e formato.

B.1. Proprietà

Si distingue tra memoria specifica e condivisa. La memoria specifica ha delle regole di uso predefinite ed è usata solo da un determinato gruppo di agenti, mentre quella condivisa è destinata a tutti gli agenti e mantiene un record di tutti i loro output.

B.2. Durata

Si classifica in memoria a breve o lungo termine. La prima è integrata negli agenti e frequentemente utilizzata nelle iterazioni multi-turno; la seconda è impiegata per conservare le informazioni importanti inerenti a compiti già eseguiti così da poterle richiamare successivamente, se necessario.

B.3. Operazioni

Le operazioni principali sulla memoria sono lettura e scrittura. Eseguendo un'operazione di lettura si punta ad ottenere delle informazioni contenute in memoria, è necessario specificare dei criteri di filtraggio e una modalità di lettura. Con la scrittura si archiviano in memoria dei dati, cercando di evitare un eventuale sovraccarico informativo.

B.4. Formato

I dati vengono salvati in memoria secondo uno specifico formato. In questa componente viene descritto il formato adottato, alcuni esempi sono: linguaggio naturale, linguaggi di programmazione, messaggi strutturati, coppie chiave-valore, embedding e strutture ad albero.

C. Percezione

Questa componente definisce le modalità con le quali l'agente riceve informazioni dall'ambiente. Le principali sono l'input testuale e l'input visivo.

C.1. Input Testuale

La maggior parte dei dati inerenti alla SE è archiviata in formato testuale. Grazie alla capacità degli LLM di interpretare il linguaggio naturale, l'input testuale è diventato il paradigma più diffuso. Esso si divide in input in linguaggio naturale e input in linguaggio di programmazione.

C.2. Input Visivo

Alcuni dati, come le interfacce utenti e i diagrammi UML, sono utili per una maggiore comprensione del contesto. Essi possono essere memorizzati in immagini e presentati tramite input visivo.

D. Azione

Con lo scopo di ampliare le capacità degli LLM la componente di azione, attraverso l'uso di strumenti, consente l'accesso a risorse esterne permettendo agli agenti di estendersi oltre la semplice interazione testuale.

D.1. Strumenti di ricerca

Strumenti che gli agenti utilizzano per ottenere informazioni. Si classificano in strumenti di ricerca web e strumenti di ricerca basati su conoscenze di base. Nel primo caso l'agente si affida ad un motore di ricerca; nel secondo caso impiega tecniche come la similarità e il confronto di stringhe per recuperare le informazioni tra le sue conoscenze base interne.

D.2. Operazioni sui file

Operazioni come modifica, ricerca, eliminazione e creazione di file possono

essere eseguite dagli agenti tramite riga di comando o librerie e strumenti fornite dai linguaggi di programmazione.

D.3. Operazioni sulla GUI

È necessario che i sistemi di agenti siano in grado di eseguire le operazioni fondamentali per interagire con le GUI data l’esistenza di diverse attività di SE relative a software per GUI.

D.4. Strumenti di testing

Comprendono strumenti per la validazione, generazione e test di mutazione. La validazione dei test ha l’obiettivo di scovare errori a runtime e test failure che possono essere usati anche come feedback per gli agenti. La generazione dei casi di test viene impiegata maggiormente grazie alla capacità degli LLM di produrre test con alto coverage in maniera poco costosa ed efficiente. Gli agenti utilizzano il testing di mutazione per verificare la correttezza dei casi di test, inoltre i risultati possono servire come feedback per test successivi.

D.5. Analisi di programmi statici.

Gli obiettivi principali per cui vengono analizzati i programmi statici sono ottenere informazioni statiche e controllare la qualità del codice per garantire una migliore comprensione del programma e una maggiore precisione delle caratteristiche del codice.

D.6. Analisi dinamica

Gli strumenti per l’analisi dinamica, come valori runtime, code coverage e tracciamento delle chiamate ai metodi, sono impiegati per analizzare e correggere il comportamento a runtime degli agenti.

D.7. Strumenti di versione di controllo

Gestiscono i cambiamenti dei file di un repository attraverso tutto il processo di sviluppo del software, utilizzati, per esempio, per mantenere aggiornata la documentazione.

D.8. Strumenti di localizzazione degli errori

Tra gli strumenti usati per la localizzazione degli errori la tecnica più diffusa è quella basata sullo spettro, adottata per individuare porzioni sospette di

codice.

2.2.3 Code Review con agenti LLM

Nell’attività di Code Review gli agenti basati su LLM simulano il processo reale di *peer review* assegnando dei ruoli differenti agli agenti. Di seguito descriviamo alcuni approcci.

CodeAgent

CodeAgent [7] è un sistema multi-agente che adotta un modello a cascata costituito da quattro fasi e un team di sei agenti, tra cui l’utente. Nella fase di sincronizzazione delle informazioni di base gli agenti che coprono il ruolo di CEO, CPO e coder discutono sulla modalità di input e sul linguaggio di programmazione da utilizzare. Successivamente, nella fase di revisione e analisi, coder e reviewer effettuano una revisione del codice e producono un report, usato per revisionare ulteriormente il codice nella fase di allineamento del codice, affidata agli stessi agenti. La fase finale è quella di documentazione, qui CEO, CTO e coder collaborano per produrre la documentazione dell’intero processo. CodeAgent è risultato particolarmente efficiente in ambiti come analisi di coerenza, vulnerabilità, verifica del formato e revisione del codice.

Metodo Rasheed et al.

Rasheed et al. [8], a differenza di CodeAgent in cui più agenti si occupano di diversi compiti, propongono un approccio in cui ogni agente è specializzato in un singolo compito. I quattro agenti (ovvero Code Review Agent, Bug Report Agent, Code Smell Agent e Code Optimization Agent) sono addestrati su dati rilevanti provenienti da GitHub e dimostrano l’elevato potenziale dei sistemi multi-agente nella revisione del codice.

ICAA

L’obiettivo del sistema ICAA [9] è identificare le incoerenze tra codice e intenzioni. Utilizza il Context & Prompt Incubation Agent per raccogliere le informazioni dal repository tramite un loop pensiero-decisione-azione. I dati raccolti vengono

analizzati, al fine di individuare le incoerenze, dal Consistency Checking Agent. Il Report Agent, infine, usa i risultati ottenuti per produrre una documentazione finale.

CORE

CORE [10] combina l’utilizzo di due agenti con strumenti tradizionali di analisi statica per identificare e correggere problemi di qualità del codice. Il Proposer Agent riceve in input i report di analisi statica insieme ai file sospetti e la documentazione degli errori per produrre in output delle versioni revisionate, successivamente filtrate mediante tool di analisi statica. Le revisioni rimanenti vengono valutate e ordinate in base alla probabilità di accettazione dal Ranker Agent.

Nei capitoli successivi proponiamo un sistema che mira a coniugare gli approcci descritti: Information Needs e impiego degli agenti basati su LLM per automatizzare la revisione del codice. Questa integrazione risponde a una limitazione fondamentale degli approcci esistenti: molti sistemi di automazione della Code Review tendono a focalizzarsi su aspetti tecnici circoscritti (come la rilevazione di bug pattern o il controllo della formattazione), tralasciando spesso quelle dimensioni contestuali e quei bisogni informativi più sottili che emergono durante una revisione umana completa. Proprio per questa ragione, l’approccio basato sugli Information Needs si rivela particolarmente promettente: fornisce una tassonomia strutturata e empiricamente validata [5] che guida l’analisi verso aspetti cruciali ma spesso trascurati, come la coerenza con le decisioni progettuali pregresse o l’appropriatezza delle soluzioni alternative. L’obiettivo del sistema è quindi quello di sfruttare la struttura offerta dagli Information Needs per guidare l’iterazione tra agente e codice, al fine di rendere la revisione più efficace e contestuale.

Metodo di Ricerca

3.1 Domande di Ricerca

La capacità di soddisfare gli Information Needs varia significativamente tra revisori umani e strumenti automatizzati. Questo studio nasce dall'esigenza di esplorare queste differenze, ponendo particolare attenzione ad alcuni aspetti chiave.

RQ1. Quali sono gli Information Needs meno soddisfatti nelle Code Review umane?

Innanzitutto, è necessario comprendere come e quali Information Needs vengono affrontati dai revisori umani durante la Code Review. Se da un lato l'esperienza umana permette di cogliere al meglio diversi aspetti contestuali, dall'altro vi sono fattori, come il tempo limitato o conoscenze specifiche, che possono influenzare in modo significativo l'efficacia del processo. Grazie a un'analisi quantitativa del coverage degli Information Needs in review umane riusciamo ad avere non solo una buona base di partenza ma anche di identificare aree critiche spesso trascurate nelle revisioni.

RQ2. Per quali specifici Information Needs gli LLM dimostrano capacità più limitate?

Il contributo offerto dagli LLM varia a seconda del tipo di Information Needs. Per aspetti tecnici ben definiti (Adeguatezza di una soluzione alternativa, Possibilità di suddivisione) dimostrano prestazioni promettenti, per aspetti più complessi (Contesto del codice) presentano capacità limitate. Valutiamo quindi la qualità delle review degli LLM in relazione a specifici Information Needs.

RQ3. Quali sono i principali punti di forza e debolezze degli agenti basati su LLM nel soddisfare i singoli Information Needs e quali differenze emergono con le review umane?

Gli LLM mostrano elevate prestazioni nell'analisi rapida di grandi volumi di dati e nell'identificazione di problemi comuni. Dimostrano, però, difficoltà in presenza di contesti complessi, casi in cui talvolta producono suggerimenti errati o trascurano informazioni cruciali. Bisogna quindi, attraverso un'analisi sistematica, verificare il grado di affidabilità degli agenti LLM. Inoltre, un'analisi per categoria permette di verificare l'esistenza di pattern interessanti, come una possibile complementarità: gli LLM riescono a coprire efficacemente aspetti tecnici puntuali, mentre le review umane sono fondamentali per valutazioni che richiedono determinati tipi di conoscenze del progetto. Le differenze cruciali riguardano la profondità dell'analisi e il tipo di feedback. Gli LLM tendono ad offrire risposte più standardizzate e commenti generici, mentre le review umane sono maggiormente contestualizzate e specifiche al problema.

Queste domande rappresentano tappe di un percorso coerente. Si parte con lo studio dello stato attuale delle revisioni umane (RQ1), in seguito si esplorano i limiti degli LLM (RQ2), per poi valutare in modo critico limiti e potenzialità al fine di confrontare i due metodi di revisione, evidenziandone differenze e complementarità (RQ3). Tale approccio graduale consente di avere una visione articolata e bilanciata del ruolo degli agenti LLM nel soddisfare gli Information Needs nella Code Review.

3.2 Pipeline di Ricerca

Per rispondere alle Domande di Ricerca, ciascuna review analizzata è stata sottoposta a un processo di analisi composto da quattro fasi principali:

- (1) Raccolta e preparazione dei dati
- (2) Categorizzazione in base agli Information Needs
- (3) Analisi mediante tool LLM
- (4) Confronto dei risultati ottenuti

La Figura 3.1 riassume i principali step del processo.

3.2.1 Raccolta e preparazione dei dati

Per questo studio sono state analizzate 66 review appartenenti a 33 progetti distinti *open-source* disponibili su GitHub. La selezione è avvenuta manualmente attraverso un processo di identificazione di Pull Request chiuse e approvate (merged) in repository pubblici. Le review sono state selezionate in modo da rappresentare una varietà di linguaggi di programmazione (Java, C, Python), tipologie di progetto, dimensione e popolarità. L'estrazione si è concentrata esclusivamente su revisioni che hanno superato positivamente il processo di code review del progetto di appartenenza. Tutte le revisioni considerate risultano approvate dai revisori.

3.2.2 Categorizzazione in base agli Information Needs

Successivamente, le revisioni sono state organizzate manualmente in base agli Information Needs soddisfatti. In caso di incertezza nella classificazione, è stato applicato un approccio di tipo *LLM-as-Evaluator*, seguendo le migliori pratiche delineate dalle *LLM Guidelines for Software Engineering* [11]. Nello specifico, è stato impiegato il modello GPT-4 di OpenAI [12] come strumento di supporto al processo decisionale, fornendogli appositi prompt strutturati per l'analisi delle review secondo la tassonomia degli Information Needs di riferimento.

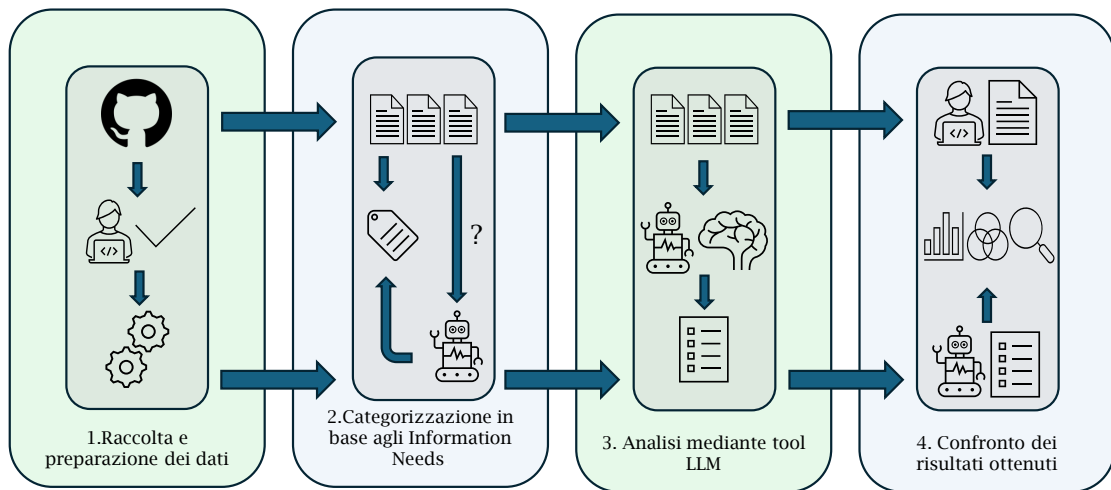


Figura 3.1: Step principali del processo di analisi di una singola code review

3.2.3 Analisi mediante tool LLM

Per svolgere questo lavoro di tesi è stato progettato e sviluppato CoRALINE, un tool sperimentale basato sul paradigma degli agenti LLM. Lo strumento è stato specificamente creato per analizzare le revisioni di codice estratte da GitHub attraverso le sue API, con il preciso obiettivo di valutare sistematicamente la capacità di soddisfare ciascuno degli Information Needs identificati in letteratura. CoRALINE genera un'analisi strutturata restituendo output in un formato predeterminato che facilita il successivo confronto con le valutazioni umane. I dettagli implementativi, le scelte architetturali e il funzionamento interno del tool sono approfonditi nella Sezione 3.3.

3.2.4 Confronto dei risultati ottenuti

Infine, è stato effettuato un confronto sistematico delle revisioni umane e quelle fornite da CoRALINE. Il confronto è stato condotto per ciascuna categoria di Information Need. Al fine di garantire un'analisi strutturata e coerente, sono state adottate le seguenti metriche qualitative:

- **Completezza:** misurare il grado di completezza di una revisione, in particolare se copre tutti gli aspetti rilevanti relativi all'Information Need esaminato.

- **Similarità:** indica il grado di sovrapposizione dei contenuti delle due tipologie di revisione, in termini di contenuti e intenzioni comunicative.
- **Profondità:** valuta il livello di dettaglio di una revisione, in particolare se gli aspetti trattati sono stati discussi in maniera approfondita o superficiale.
- **Errori:** identifica i casi in cui CoRALINE ha generato analisi errate, includendo errori semantici o casi particolari, come *hallucination*.

3.3 CoRALINE

CoRALINE (Code Review Agents with LLM for Information Needs Evaluation) è un sistema *multi-agent* e *multi-task*, in cui ciascun agente utilizza un modello LLM per eseguire l'analisi rispetto a uno specifico Information Needs della code review fornita come input. Gli agenti lavorano in parallelo e ciascuno produce una porzione strutturata dell'output finale.

3.3.1 Scelte progettuali

L'architettura di CoRALINE è stata progettata con l'obiettivo di bilanciare precisione analitica, efficienza computazionale e flessibilità nell'integrazione con altri sistemi. Per garantire tali esigenze, le scelte implementative si sono concentrate su tre aspetti principali: scelta del modello LLM, modalità di gestione del comportamento degli agenti rispetto agli Information Needs, modalità di gestione del flusso di dati.

Scelta modello LLM

La selezione del modello per il funzionamento degli agenti rappresenta una decisione critica in quanto le sue prestazioni influenzano tutto il sistema, dai tempi di esecuzione alla qualità delle risposte generate. La scelta è ricaduta su CodeLlama:instruct, una variante *fine-tuned* di CodeLlama [13] ottimizzata per task di comprensione e generazione di codice seguendo istruzioni naturali. Le caratteristiche principali che hanno motivato la scelta di tale modello sono:

- **Specializzazione nel dominio del codice:** CodeLlama:instruct è addestrato su dataset con alto rapporto codice/testo, a differenza di altri LLM generici, garantendo una comprensione più approfondita delle strutture semantiche e sintattiche.
- **Capacità di interpretare istruzioni complesse:** la variante *instruct* è addestrata per comprendere comandi articolati e gerarchie di task, ideale per soddisfare i diversi Information Needs degli agenti in parallelo.
- **Disponibilità di versioni scalabili:** il bilanciamento tra le dimensioni del modello (7B/13B/34B parametri) consente di eseguire più istanze contemporaneamente senza *overhead*, aspetto di grande rilevanza nei sistemi multi-agente.
- **Gestione di finestre di contesto estese:** la capacità di gestire input di lunghezza elevata consente di analizzare revisioni ampie senza perdita di coerenza nelle risposte.

Gestione agenti e Information Needs

Il problema centrale del sistema è come istruire un agente per soddisfare gli Information Needs. La soluzione adottata prevede l'uso di un singolo agente per un determinato Information Need. Durante la progettazione, due Information Needs sono stati esclusi a seguito di analisi empiriche:

- **Motivazione:** l'agente incaricato di verificare l'assenza di informazioni nella pull request (ad esempio, la giustificazione di una modifica) non è riuscito a produrre valutazioni affidabili. Il LLM, infatti, tendeva a generare falsi positivi (*hallucinations*) o a ignorare criteri contestuali essenziali. Poiché l'analisi della "Motivazione" richiede una comprensione profonda delle logiche progettuali – spesso implicite o disperse nel repository – si è optato per rimuovere questo agente, delegando tale controllo ai revisori umani.
- **Competenze specialistiche:** richiedere l'intervento di un utente specifico contraddice l'obiettivo di automazione *end-to-end* del sistema. Tale approccio si è rivelato un anti-pattern nell'automazione del processo, in quanto violava il principio di *self-containment* del sistema.

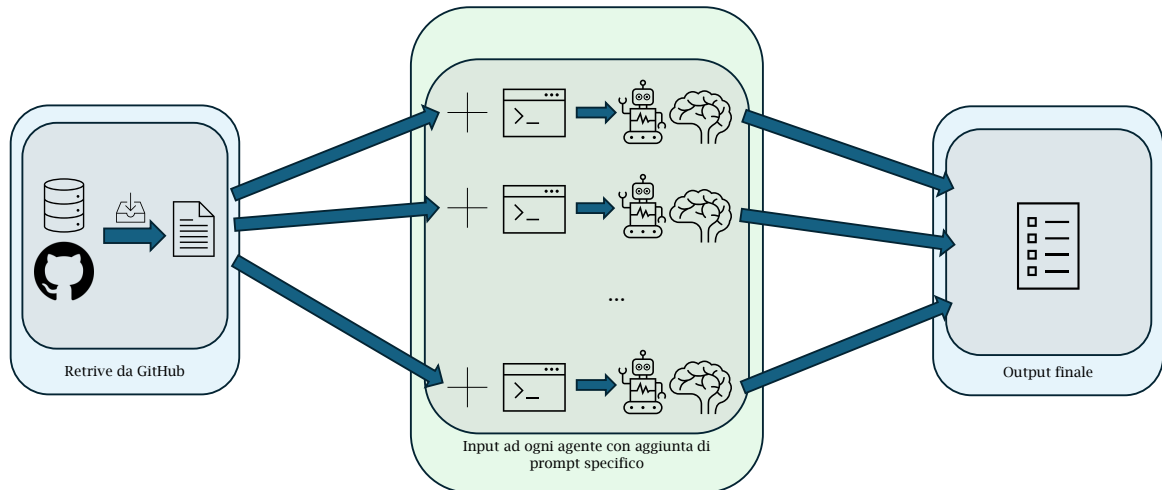


Figura 3.2: Rappresentazione grafica del processo di gestione del flusso dati eseguito da CoRALINE

In parallelo l'Information Need **Contesto del codice** è stato suddiviso in **Contesto del codice** e **Contesto della descrizione**. Inizialmente un solo agente si occupava di entrambi i compiti, ma i risultati ottenuti hanno evidenziato scarsa profondità di analisi e precisione, motivo per cui si è deciso di suddividere, ottenendo miglioramenti significativi.

Prompt design e struttura del prompt

L'uso di un singolo agente per un determinato Information Need permette di usare una tecnica di *prompt engineering* specifica per ogni agente, consentendo un'analisi più precisa, evitando la propagazione di errori tra gli agenti e garantendo una maggiore scalabilità. Per garantire l'efficacia, è stato applicato un procedimento iterativo di raffinamento dei prompt. La struttura del prompt per ogni agente segue uno schema standardizzato composto da due messaggi:

- **Messaggio di sistema:** contiene il contesto operativo dell'agente, includendo il suo ruolo specifico, relativo all'Information Needs di competenza
- **Messaggio utente:** contiene i dati specifici della review estratti dalle API di GitHub.

Gestione del flusso di dati

Come mostrato in Figura 3.2, per gestire il flusso di dati il sistema implementa un processo strutturato volto a garantire efficienza e coerenza dell'analisi. Il tool si interfaccia con le API di GitHub per ottenere le informazioni necessarie relative a una *pull request*, in particolare: cambiamenti nel codice, descrizione della *pull request* e del progetto. Ogni agente riceve gli stessi dati insieme a un prompt specifico per il suo tipo di analisi. Gli agenti lavorano in modo indipendente e in parallelo, producendo un proprio output. Al termine dell'esecuzione di tutti gli agenti, viene generato un documento di testo standardizzato, strutturato secondo un formato definito, contenente i risultati prodotti da ogni agente. Inoltre, il sistema offre la possibilità di eseguire un'analisi completa di tutti gli Information Needs oppure un'analisi parziale di un singolo Information Need, garantendo flessibilità e riducendo i tempi di esecuzione, permettendo l'adattamento a diversi scenari d'uso. Un esempio di output di CoRALINE è mostrato nella Figura 3.3.

Suitability of an alternative solution:
The code you provided works as intended, but there are a few potential improvements that could make it more efficient or easier to read: 1. Use <code>`equals()`</code> instead of using <code>`==`</code> to compare strings. This would prevent the <code>"=="</code> operator from throwing an exception if one of the operands is null. 2. Initialize variables on declaration; this makes the code easier to read and avoids mistakes caused by forgetting to initialize a variable before using it. ...
Correct understanding:
The code is well-formatted and easy to read. Here are some suggestions that can make the code even more understandable: 1. Use descriptive variable names: Many of the variables have generic names such as <code>'action'</code>, <code>'collectionName'</code>, etc. Using more descriptive names can help identify what each variable refers to without having to examine their types or contexts. For example, <code>'activeCollection'</code> instead of <code>'action'</code> for a collection object that is currently being operated on. 2. Use comment blocks: The code doesn't have a lot of comments yet. Adding comment blocks above functions and class definitions can provide more information about what those pieces of ...
Code Description:
... Overall, I believe the patch adds relevant functionality that matches the description. It is well-organized and follows best practices for coding, with clear naming conventions and meaningful comments. However, there might be a few small issues to watch out for, such as formatting inconsistencies or unnecessary dependencies included in the pull request. ...
Code Context:
This code snippet seems to be an excerpt from the "obs-studio" project's source code. The code appears to be related to the management of scene collections used in OBS Studio (Open Broadcasting Software). Specifically, this code appears to be responsible for populating a menu with available scene collections and handling user input when selecting a new scene collection. The code uses Qt libraries to create GUI elements and handle events. ...
Necessity:
Here are some recommendations for simplifying and omitting parts of the code: 1. Simplify profile setup UI: Instead of rendering the entire list of profile options each time a new profile is created, you could render only the necessary options based on the selected platform and device. For example, if you select "Android", then only display options related to Android profiles (e.g., API level, SDK version, etc.). This will simplify the UI and reduce the amount of code that needs to be rendered each time a new profile is created. 2. Omitting unnecessary checks: The implementation contains redundant checks for whether the selected platform and device are valid or not. These checks can be omitted if you trust the user input to always contain correct values. However, it's important to ensure that any subsequent code relies on valid user input, as incorrect data could lead to unexpected behavior. ...
Atomicity:
... As such, you should consider making your changes atomic if they are going to affect shared resources. ...

Figura 3.3: Esempio di una revisione di codice fornita in output dal tool CoRALINE

Analisi dei Risultati

4.1 Introduzione

Presentiamo i risultati sperimentali ottenuti da un'analisi individuale delle review umane e dal confronto sistematico con le review generate dal tool CoRALINE basato su LLM. L'obiettivo è valutare l'efficacia dei modelli di linguaggio nella revisione del codice seguendo dei bisogni informativi ben definiti. I risultati sono organizzati seguendo la struttura delle quattro *Research Question* formulate nel Capitolo 3.

4.2 Metodologia di analisi

Per garantire un'analisi sistematica e replicabile, la valutazione comparativa tra revisioni umane e quelle generate da CoRALINE è stata condotta applicando le metriche descritte nella Sezione 3.2.4 a ciascuna review:

- **Completezza:** misurata attraverso un'ispezione manuale che verificava la presenza o assenza di elementi riconducibili a ciascun Information Need. Per ogni review, è stato assegnato un valore binario (SI/NO) per indicare se l'Information Need era stato soddisfatto o meno.

- **Similarità e Profondità:** valutate applicando un approccio *LLM-as-Evaluator*. Il modello GPT-4 [12] è stato incaricato di confrontare le coppie di review (umana/LLM) ai fini di valutare il loro grado di sovrapposizione nei contenuti e nelle intenzioni comunicative, il livello di dettaglio e la pertinenza dei suggerimenti. La valutazione si basa sull'analisi semantica della descrizione fornita dal modello.
- **Errori:** attraverso un processo di labeling manuale, sono stati identificati e classificati gli errori nelle revisioni di CoRALINE in tre categorie mutuamente esclusive: hallucination (suggerimenti inventati o fuorvianti), inconsistenze logiche (raccomandazioni in contraddizione con la logica del codice o le best practice note), e incomprensioni del contesto (errori derivanti dalla mancanza di informazioni contestuali non presenti nell'input).

4.3 Risultati sperimentali

Questa sezione presenta i risultati emersi dall'analisi comparativa tra revisioni umane e automatizzate. I dati sono organizzati secondo tre prospettive complementari che corrispondono agli obiettivi della ricerca: (1) la copertura degli Information Needs nelle revisioni umane, (2) le prestazioni degli LLM rispetto ai diversi bisogni informativi, (3) i punti di forza e i limiti degli agenti basati su LLM e le differenze sostanziali tra i due approcci.

4.3.1 RQ1. Quali sono gli Information Needs meno soddisfatti nelle Code Review umane?

L'analisi per la RQ1 è stata condotta attraverso un conteggio sistematico della frequenza con cui ciascun Information Need risultava soddisfatto nelle revisioni umane esaminate. Per ogni review, è stato valutato se contenesse elementi riconducibili a ciascuna delle categorie della tassonomia, registrandone la presenza o assenza. La Figura 4.1 mostra la copertura degli Information Needs nelle review umane analizzate. È evidente una marcata disparità nella frequenza con cui tali bisogni vengono affrontati: in particolare, possiamo notare come **Possibilità di suddivisione**

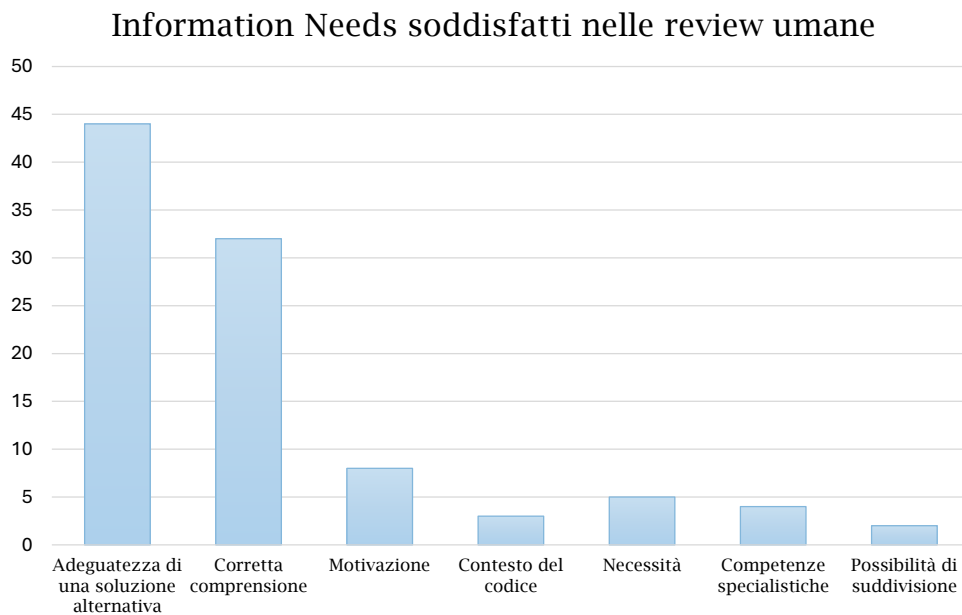


Figura 4.1: Copertura degli Information Needs nelle review umane analizzate

e **Contesto del codice** risultino significativamente più trascurati. I risultati emersi da questa analisi sono in linea con quanto osservato nello studio di Pascarella et al.[5], il quale ipotizza che la scarsa attenzione verso tali categorie sia dovuta al fatto che i revisori non percepiscano sempre la necessità di acquisire determinate informazioni. **Adeguatezza di una soluzione alternativa** risulta essere l'Information Need più frequentemente affrontato, a dimostrazione del fatto che i revisori tendono a concentrarsi maggiormente su aspetti legati all'ottimizzazione e all'efficacia del codice. Questa disomogeneità nella copertura dei bisogni informativi evidenzia lacune che possono compromettere in modo significativo la qualità e l'efficacia del processo. In tale contesto, approcci alternativi, come quello basato su LLM, acquisiscono particolare rilevanza, in quanto potenzialmente in grado di colmare tali mancanze.

4.3.2 RQ2. Per quali specifici Information Needs gli LLM dimostrano capacità più limitate?

L'analisi dei risultati ha evidenziato un netto trade-off tra le capacità degli LLM e dei revisori umani nel soddisfare i diversi Information Needs, determinato dalla natura del compito richiesto. La Figura 4.2 sintetizza questo bilancio, mostrando in quali Information Needs l'analisi automatizzata tende a essere più dettagliata di

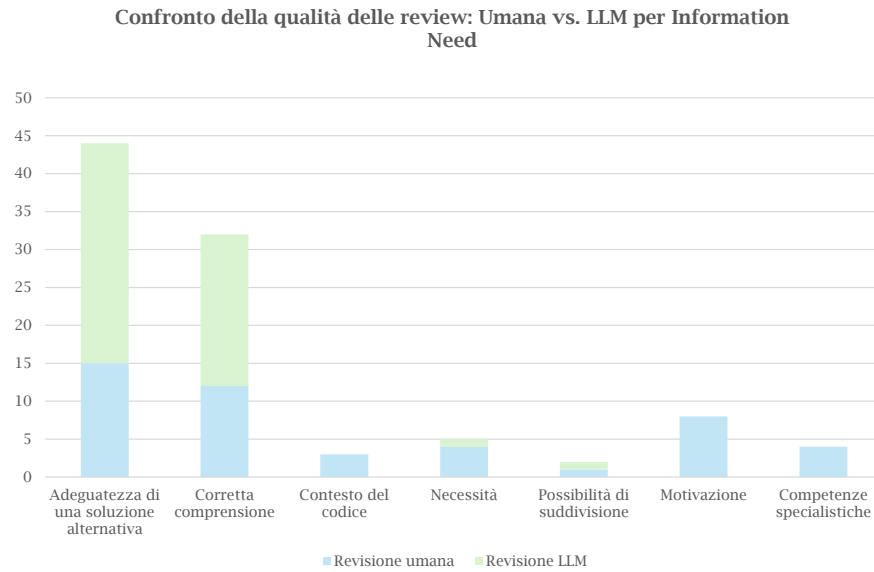


Figura 4.2: Distribuzione, per categoria di Information Need, delle revisioni in cui l’analisi umana o quella automatizzata (LLM) è stata valutata come più dettagliata.

quella umana e viceversa. I dati rivelano che gli LLM eccellono nel fornire un’analisi quantitativamente più ricca e sistematica per bisogni informativi ben definiti e tecnici. Al contrario, le limitazioni degli LLM emergono in modo marcato laddove è richiesta una comprensione approfondita del contesto. L’analisi degli errori fornisce evidenze chiare di queste limitazioni. Come illustrato in Figura 4.3, la frequenza e la tipologia di errori commessi da CoRALINE variano in base alla natura dell’Information Need. L’analisi del Contesto del codice e della descrizione ha registrato i tassi di errore più elevati, prevalentemente dovuti a incomprensioni del contesto. Questo conferma che le capacità degli LLM sono fortemente vincolate alla completezza e alla qualità delle informazioni contestuali fornite in input. Gli LLM mostrano limiti evidenti nei casi in cui è richiesta una comprensione approfondita del contesto. In particolare, ciò accade quando l’analisi dipende da elementi non inclusi nell’input, come riferimenti ad altre *pull request*, linee guida di progetto o convenzioni di sviluppo. In tali situazioni, il modello di linguaggio tende a fornire risposte generiche o, talvolta, imprecise. La mancanza di un contesto ampio limita in modo significativo le prestazioni dei LLM, a differenza dei revisori umani che possono contare su una comprensione olistica del progetto e su conoscenze implicite difficilmente accessibili al modello. Al contrario, per Information Needs più tecnici e auto-contenuti, come **Adeguatezza di una solu-**

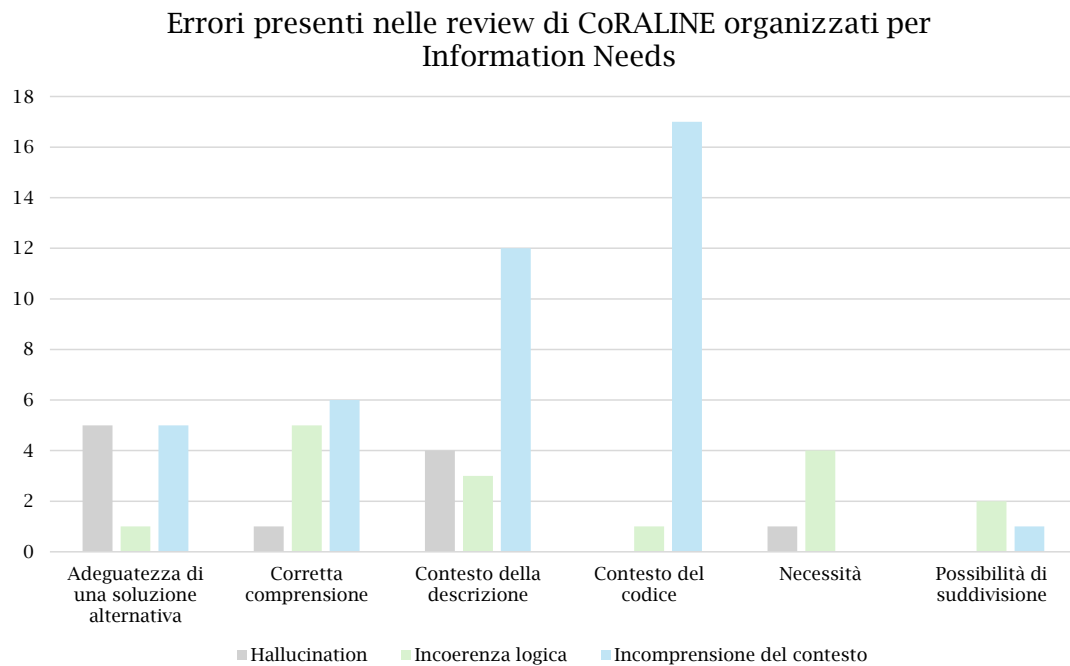


Figura 4.3: Frequenza e tipologia di errori individuati nell’analisi dei singoli Information Needs con il tool CoRALINE.

zione alternativa e **Possibilità di suddivisione**, gli LLM mostrano risultati notevoli, riuscendo a fornire analisi rapide e, in diversi casi, a identificare problematiche che potrebbero sfuggire anche a un revisore umano.

4.3.3 RQ3. Quali sono i principali punti di forza e debolezze degli agenti basati su LLM nel soddisfare i singoli Information Needs e quali differenze emergono con le review umane?

Di seguito presentiamo i punti di forza e le debolezze di ciascun agente, in relazione all’analisi del relativo Information Needs. La Figura 4.4 presenta una versione abbreviata di tali risultati.

Adeguatezza di una soluzione alternativa: l’agente eccelle nel proporre ottimizzazioni, spesso trascurate dai revisori umani, grazie alla sua vasta conoscenza di pattern e best practice. Tale flessibilità si trasforma in limite quando i suggerimenti ignorano lo stato effettivo del codice. In alcuni casi, l’agente ha proposto modifiche teoricamente valide ma potenzialmente destabilizzanti per equilibri preesistenti

non documentati (es. codice funzionante ma non chiaro che il modello tenta di migliorare rischiando regressioni). I revisori umani, invece, forniscono alternative mirate alla specifica sezione di codice, considerando l'architettura complessiva del progetto.

Corretta comprensione: i risultati mostrano che l'agente si distingue nel garantire leggibilità e aderenza a standard formali, indicando sistematicamente l'assenza di commenti o test insufficienti. Tale sistematicità, tuttavia, genera ridondanza, con suggerimenti ripetitivi talvolta su aspetti marginali, come funzioni banali, riducendo l'utilità complessiva. A differenza degli LLM, i revisori umani riescono ad operare a un livello di astrazione superiore, trattando aspetti logici del programma più intrinseci. Ad esempio, il LLM tende a suggerire di inserire nomi di variabili più descrittivi, mentre un revisore umano potrebbe interrogarsi sul motivo della scelta di un determinato tipo di variabile invece che un altro.

Motivazione: tale Information Need, sebbene non incluso nello studio con il tool CoRALINE, rappresenta una sfida intrinseca, anche con un contesto esteso gli LLM faticano a comprendere logiche progettuali implicite o decisioni storiche non tracciate, richiedendo inevitabilmente l'intervento dei revisori umani. Questi ultimi possiedono conoscenze specifiche sullo sviluppo del progetto, che consentono loro di valutare se le modifiche proposte siano coerenti o meno ai fini del progetto.

Information Needs	Punti di forza	Debolezze
Adeguatezza di una soluzione alternativa	Propone ottimizzazioni creative e alternative non ovvie	Suggerimenti a volte ignorano modifiche preesistenti o equilibri non documentati, rischiando regressioni.
Corretta Comprensione	Migliora leggibilità e aderenza a standard formali (commenti, test).	Tendenza a suggerimenti ridondanti su aspetti marginali
Motivazione		Difficoltà nel comprendere logiche progettuali implicite o decisioni storiche non documentate
Contesto del codice	Buone capacità nell'identificazione delle incoerenze	Dipendenza dalla completezza del contesto in input
Necessità	Rileva codice ridondante e suggerisce alternative efficienti.	Falsi positivi su ridondanze deliberate
Competenza specialistiche		Richiedere intervento umano viola il principio di automazione end-to-end
Possibilità di suddivisione	Analisi precisa dell'autonomia delle modifiche; suggerimenti chiari per separarle	Difficoltà nel riconoscere accorpamenti intenzionali

Figura 4.4: Tabella riassuntiva dei punti di forza e delle debolezze di ciascun agente nel soddisfare il relativo Information Need.

Contesto del codice: l'agente mostra buone capacità nell'identificazione di incoerenze, ma la qualità dell'analisi è fortemente vincolata alla completezza delle informazioni fornite. Modifiche marginali ma strategicamente rilevanti possono essere erroneamente contrassegnate come "non coerenti" se il contesto fornito è parziale. Ad esempio, l'introduzione di un parametro opzionale potrebbe essere considerata incoerente se non menzionato nella descrizione della *pull request*. Un revisore umano, invece, avendo accesso all'intero contesto del codice, saprebbe riconoscere che tale aggiunta è stata introdotta per motivi di retrocompatibilità.

Necessità: l'agente dimostra ottime capacità nel rilevare codice ridondante, suggerendo alternative più efficienti; nonostante ciò, la mancanza di visibilità sull'intero progetto lo porta a falsi positivi, classificando come "superflue" ridondanze intenzionali (es. duplicazioni per motivi di retrocompatibilità). Errori di questo tipo sono difficilmente commessi dai revisori umani.

Competenze specialistiche: è stato escluso dal sistema in quanto antitetico all'automazione end-to-end: delegare al LLM la richiesta di intervento umano introdurrebbe un anti-pattern, rompendo il principio di self-containment e creando dipendenze esterne. Al contrario, i revisori sanno perfettamente a chi rivolgersi, sfruttando dinamiche collaborative.

Possibilità di suddivisione: l'agente offre analisi precise sull'autonomia delle modifiche, proponendo divisioni ottimali quando necessario. Tuttavia, non distingue tra mancanza di modularità e scelte intenzionali (es. modifiche accorpate per coerenza logica), rivelando un gap nell'interpretazione delle dinamiche collaborative umane. I revisori umani, infatti, tendono a mantenere modifiche accorpate per ragioni logistiche, anche se non perfettamente modulari.

4.4 Discussione critica

I risultati evidenziano una complementarità di funzione tra i revisori umani e gli LLM nell'ambito della revisione del codice. Gli LLM forniscono volume e sistematicità, garantendo che tutti gli Information Needs vengano almeno considerati, anche

quelli spesso trascurati dai revisori. Tuttavia, come dimostrato dalla metrica degli errori, il loro output richiede un filtro umano per la validazione contestuale. Il caso dell'Information Need "Adeguatezza di una soluzione alternativa" ben esemplifica questa sinergia. Da un lato l'LLM funge da generatore di idee che, pur contenendo suggerimenti potenzialmente destabilizzanti, forniscono al revisore umano un ventaglio di alternative potenziali a cui egli non avrebbe altrimenti pensato. Il revisore, a sua volta, applica la sua comprensione del contesto per scartare le alternative non applicabili e selezionare o ispirare quelle valide. In questo modello, l'LLM non sostituisce il giudizio umano, ma potenzia la fase esplorativa della revisione, mentre l'umano rimane indispensabile per la fase decisionale finale. I dati sulla frequenza e tipologia di errori dimostrano in modo inequivocabile come le prestazioni degli LLM degradino in modo significativo quando l'analisi richiede una comprensione olistica del progetto o l'integrazione di conoscenze non direttamente presenti nel codice sotto revisione. La predominanza di "incomprensioni del contesto" come categoria di errore principale è la prova tangibile di questo limite fondamentale. Gli LLM presentano quindi limiti significativi legati alla loro dipendenza dal contesto fornito in input e alla natura sistematica delle loro risposte. Inoltre, la loro abilità di analizzare rapidamente ampie porzioni di codice e soddisfare diversi bisogni informativi è messa in discussione dalla tendenza a produrre risposte generiche e poco adatte a situazioni complesse o non esplicitamente documentate. Tale efficienza è direttamente correlata alla completezza dei dati forniti in input e alle capacità intrinseche del modello adottato. In sintesi, l'integrazione tra revisione umana e automatizzata potrebbe rappresentare un approccio promettente per coniugare profondità analitica ed efficienza operativa. Tutto il materiale utilizzato e prodotto negli esperimenti è disponibile online.¹

¹Replication Package:<https://github.com/Giuliopalla/CoRALINE--Tool-e-Risultati>

Conclusioni e Sviluppi futuri

Questa tesi ha presentato un'analisi sistematica dell'impiego degli LLM nell'attività di Code Review, ponendo particolare attenzione alla loro capacità di soddisfare gli Information Needs. Lo studio è stato articolato in quattro fasi:

- Raccolta e preparazione di revisioni umane da GitHub.
- Categorizzazione delle revisioni umane in base agli Information Needs.
- Analisi delle revisioni mediante un tool basato su agenti LLM, progettato per analizzare ciascun Information Needs in modo indipendente.
- Confronto e validazione delle differenze tra le due tipologie di revisione.

I risultati hanno confermato che gli LLM coprono tutti gli IN in modo sistematico, ma con suggerimenti spesso generici o dipendenti dal contesto fornito, mentre le revisioni umane, sebbene più limitate in termini di copertura, offrono analisi mirate e contestualizzate. I limiti emersi aprono diverse direzioni per ricerche future, tra cui l'impiego di modelli con finestre di contesto più ampie, l'adozione di tecniche avanzate di prompt engineering (come *Chain-of-Thought*), l'uso di agenti specializzati che integrano modelli differenti a seconda del tipo di analisi, o lo sviluppo di sistemi che incorporano feedback attraverso approcci *human-in-the-loop*. Si raccomanda infine

l'utilizzo degli LLM come primo filtro automatizzato nella Code Review, ai fini di un'identificazione rapida dei problemi tecnici, a una riduzione del carico cognitivo del revisore e a porre attenzione ad aspetti spesso trascurati nelle revisioni. Tuttavia, è importante evitare di affidare loro valutazioni che richiedono un'ampia comprensione del contesto o dinamiche collaborative.

Bibliografia

- [1] M. Chen, J. Tworek, and et al., “Evaluating large language models trained on code,” 2021. (Citato a pagina 1)
- [2] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, “The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects,” 2014. (Citato a pagina 1)
- [3] A. Alami, V. V. Jensen, and N. A. Ernst, “Accountability in code review: The role of intrinsic drivers and the impact of llms,” 2025. (Citato alle pagine 1 e 5)
- [4] Z. Yang, C. Gao, Z. Guo, Z. Li, K. Liu, X. Xia, and Y. Zhou, “A survey on modern code review: Progresses, challenges and opportunities,” 2024. (Citato a pagina 5)
- [5] L. Pascarella, D. Spadini, F. Palomba, M. Bruntink, and A. Bacchelli, “Information needs in contemporary code review,” 2018. (Citato alle pagine 6, 15 e 27)
- [6] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” 2024. (Citato a pagina 9)
- [7] D. Tang, Z. Chen, K. Kim, Y. Song, H. Tian, S. Ezzini, Y. Huang, J. Klein, and T. F. Bissyandé, “Codeagent: Collaborative agents for software engineering,” *CoRR*, vol. abs/2402.02172, 2024. (Citato a pagina 14)

- [8] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, "Ai-powered code review with llms: Early results," *CoRR*, vol. abs/2404.18496, 2024. (Citato a pagina 14)
- [9] G. Fan, X. Xie, X. Zheng, Y. Liang, and P. Di, "Static code analysis in the ai era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents," *CoRR*, vol. abs/2310.08837, 2023. (Citato a pagina 14)
- [10] N. Wadhwa, J. Pradhan, A. Sonwane, S. P. Sahu, N. Natarajan, A. Kanade, S. Parthasarathy, and S. K. Rajamani, "Frustrated with code quality issues? llms can help!" *CoRR*, vol. abs/2309.12938, 2023. (Citato a pagina 15)
- [11] S. Wagner, M. Muñoz Barón, D. Falessi, and S. Baltes, "Towards evaluation guidelines for empirical studies involving llms," 2024. (Citato a pagina 18)
- [12] OpenAI, "Chatgpt (version gpt-4.5) [large language model]," <https://openai.com/chatgpt>, 2024. (Citato alle pagine 18 e 26)
- [13] M. AI, "Code llama:instruct [large language model]," <https://ai.meta.com/blog/code-llama-large-language-model-coding/>, 2023. (Citato a pagina 20)

Ringraziamenti

Ringraziamenti qui...